

P-CORE: Self-Supervised Surface Consistency for Point-Based Neural Editing

Yanshu Zhang¹, Shichong Peng¹, Mehran Aghabozorgi¹, Alireza Moazeni¹, and Ke Li¹

Simon Fraser University

Abstract. Advances in neural rendering have enabled high-fidelity multi-view reconstruction of 3D scenes. However, free-form non-rigid shape editing remains a significant challenge. Point-based neural representations are highly desirable for multi-view reconstruction because they lack fixed connectivity, which does not constrain the learned surface topology to that of the initialization. Yet this same property causes point-based representations to struggle with holes and surface discontinuities under large deformations. To address this, we propose a novel self-supervised method to enable point-based representations to adapt to large deformations without requiring ground truth multi-view images of deformed geometry. The key idea is to generate random deformations and to ensure consistency in the predicted surface before and after deformation. In particular, the surface prediction from the deformed point cloud should be the same as the deformation applied to the surface prediction from the original point cloud. We incorporate our approach into attention-based point representations, which differ from splatting-based point representations in their use of a learned interpolation kernel between points as opposed to a Gaussian kernel around each point. This learned interpolation kernel can learn to adapt to large deformations, without requiring addition or removal of points. We show that our framework significantly enhances its robustness to large deformations. Experiments on synthetic geometry editing benchmarks (Neural Editor, Objaverse) demonstrate that our approach outperforms existing point-based methods in zero-shot editing and significantly reduces artifacts. Furthermore, qualitative results on the DTU and Mip-NeRF 360 datasets demonstrate our method’s effectiveness on real-world scenes.

1 Introduction

Achieving free-form non-rigid editing of photo-realistic neural scenes is highly desirable for applications in animation, design, and gaming. Recent advances in neural rendering [37, 47, 50, 69, 77] have enabled high-fidelity multi-view reconstructions, but performing *zero-shot* deformation of these reconstructed scenes remains an open challenge. Point-based neural renderers have emerged as a highly desirable representation for reconstruction; by lacking fixed connectivity, they do not constrain the learned surface topology to that of the initialization, offering exceptional flexibility in capturing high-fidelity details of complex structures.

However, when transitioning from static reconstruction to free-form non-rigid editing, this lack of connectivity becomes a double-edged sword. When the point cloud is deformed, the points drift apart because there are no explicit edges or polygons holding them together. Without explicit topological connections, point-based methods severely struggle with holes, see-through artifacts, and surface discontinuities under large deformations. To tackle these issues, existing approaches often rely on explicit spatial regularizations [26, 30, 62] or reintroduce proxy geometry such as cages [29, 31, 53, 70] or meshes [23–25, 32, 45, 65, 66, 71, 74, 80]. While these proxies can enforce structural integrity, they introduce a host of new limitations. Constructing an artifact-free proxy that tightly encloses the geometry without self-intersections is complex and often requires manual intervention. Furthermore, proxy-based methods are restricted to edits that the rigid topology can support, making part-level manipulations extremely challenging.

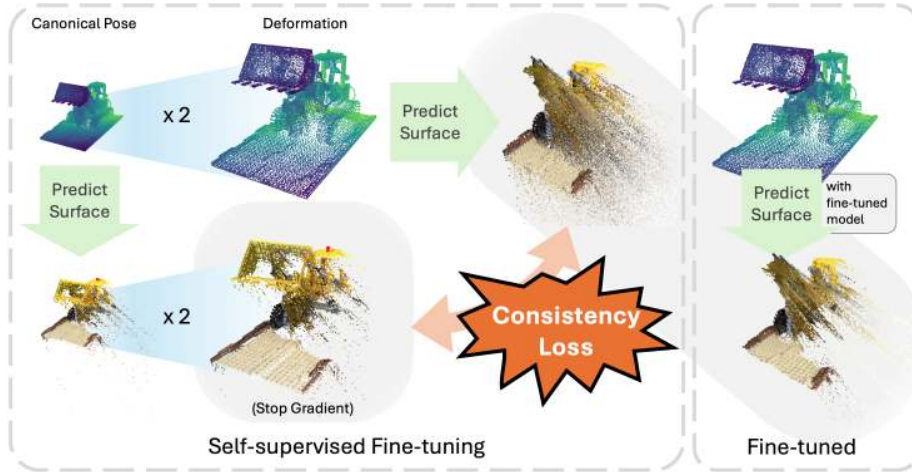


Fig. 1: Overview. As shown in the top left, while point-based representations can learn a good surface prediction from multi-view images in its canonical space, they often struggle with holes and surface discontinuities during unseen deformations. To address this, we propose a self-supervised surface consistency framework. Given a point cloud representation learned in the canonical space, we predict the surface points from it. We then apply an identical deformation field to both the canonical point cloud and these extracted surface points. The resulting deformed surface points serve as geometric pseudo-ground truth to supervise the model’s new surface predictions on the deformed point cloud. A stop-gradient operation is applied to the target points to prevent model collapse. Our fine-tuning process significantly refines the surface predictions, ensuring robust geometric consistency under severe deformations.

To overcome these limitations without sacrificing the flexibility of point-based representations, we introduce a novel **self-supervised surface consistency framework**, as illustrated in Figure 1. Instead of relying on ground truth de-

formed meshes or manually crafted spatial regularizations, our pipeline generates random deformations and enforces a surface consistency constraint: the surface predicted from the deformed point cloud should equal the deformation applied to the surface predicted from the original point cloud. By generating such geometric “pseudo-ground truth” for the deformed state and jointly enforcing photometric anchoring on the original state, our method significantly improves the generalizability of point-based renderers to unseen edits, ensuring hole-free, smooth surface reconstruction and high-fidelity rendering under non-rigid deformations.

We deliberately apply this framework to Proximity Attention Point Rendering (PAPR) [77], an attention-based neural point renderer. Unlike 3D Gaussian Splatting (3DGS) [37], where discrete, anisotropic Gaussians must be coherently transformed (e.g., adjusting covariances) to avoid gaps and tears after deformation, PAPR treats each point as infinitesimal and fills gaps via a learned attention-based interpolation kernel between nearby points. This design is advantageous for two reasons: first, it eliminates the need to transform spatial extents under deformation, inherently supporting continuous surface prediction; second, the learned interpolation kernel can adapt to large deformations without requiring addition or removal of points, offering the capacity to generalize to unseen edits. We demonstrate that when coupled with our self-supervised framework, PAPR’s robustness to large deformations is significantly enhanced, preventing topological breakage even under drastic non-rigid edits.

Contributions Below are our key contributions:

- We propose a novel self-supervised fine-tuning framework that substantially reduces holes and surface discontinuities in point-based neural renderers without requiring ground-truth dynamic data or geometry proxies.
- We demonstrate the successful coupling of this framework with an attention-based neural point renderer (PAPR), highlighting its inherent advantages over spatial-extent-based methods (e.g., 3DGS) for continuous surface prediction during large deformations.
- We provide results on synthetic geometry editing benchmarks (Neural Editor, Objaverse) demonstrating that our method outperforms both proxy-based and proxy-free point-based methods.

2 Related Work

2.1 3D Shape Deformation

There is a long line of work in geometry processing on editing 3D shapes [21, 73]. Many surface-based methods use parametric patches or surface meshes as proxies to manipulate shapes [4, 15, 19, 20, 33, 34, 38, 58, 64], utilizing approaches like Laplacian [22, 44, 59–61] and cage-based [34, 72, 78] deformations. However, designing an appropriate proxy (e.g., mesh or cage) that closely fits the target model is challenging. Manual creation can be time-consuming and may require extensive expertise, while automatic generation methods might not always produce optimal proxies, especially for complex or highly detailed models.

To bypass proxy construction, traditional point-based deformation methods directly manipulate the shape using a set of freely positioned points or Moving Least Squares (MLS) handles [2, 5–8, 10–12, 27, 28, 41, 48, 49, 54, 57, 82]. While offering greater flexibility, directly applying these traditional point editing algorithms to modern neural point renderers introduces a critical failure mode. Traditional algorithms only relocate the discrete 3D coordinates. Because point clouds lack connectivity, deformations that stretch or bend the shape cause the points to spread apart. In neural renderers like 3DGS, this naïve relocation breaks the continuous volumetric density implied by the canonical point distribution and extents (e.g., covariances), leading to severe holes, see-through artifacts, and surface tearing. The neural representation is no longer consistent with the displaced geometry.

To address this in neural rendering, recent approaches [24, 31, 53, 66, 70, 71, 74, 80] fall back to extracting and editing a proxy geometry (mesh or cage) to maintain surface continuity, and then map the edits back to the neural scene. However, this conversion introduces approximation errors and additional points of failure, because some geometric features easily represented implicitly cannot be represented in a proxy, and vice versa. Moreover, the edits are limited to what the rigid proxy connectivity supports—part-level editing (e.g., spinning wheels) becomes severely challenging.

Another line of work [9, 18, 30, 68, 79] trains neural representations on dynamic scene observations to learn shape variations over time. They associate each shape with keypoints, allowing the shape to change when keypoints are edited. However, these methods fail to generalize to out-of-distribution (OOD) shapes and edits not observed in the dynamic scenes. In contrast, our method operates directly on points to preserve flexibility without requiring dynamic scene observations, and introduces a self-supervised surface consistency framework to fundamentally resolve the surface tearing and hole artifacts caused by point spreading under deformation.

2.2 Point-based Neural Rendering

Point-based representations [37, 69, 77] have gained significant attention in neural rendering. Some methods [3, 39, 51, 55] assume a given point cloud from Multi-View Stereo (MVS) or LiDAR, while others learn it from an initialization. Since point clouds natively lack connectivity, a fundamental challenge is how to fill the gaps between discrete points. One line of work focuses on predicting continuous surfaces from point clouds, employing classical techniques like Moving Least Squares (MLS) [1, 43] and Poisson Surface Reconstruction [35, 36], or more recent learning-based implicit surface prediction methods [17, 32, 42, 46]. Another line of work bypasses explicit meshing and directly renders the points. To fill the gaps during rendering, most methods associate a spatial extent with each point, either using 2D splats [40, 56, 67, 75, 81] or 3D Gaussian splats [37]. While these spatial extents enable high-quality rendering, they introduce parameters (e.g., covariances) that are difficult to correctly adjust during deformation. Instead, methods like PAPR [77] and Pointersect [13] represent points without spatial

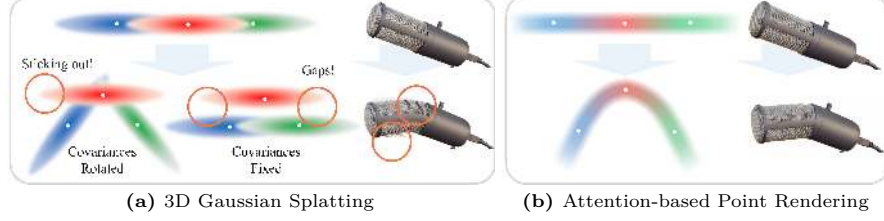


Fig. 2: As shown on the left, in 3DGS editing the points requires adjusting each Gaussian splat’s covariance to eliminate gaps. However, this adjustment may introduce distortions, as circled. Instead, attention-based methods model only the center of each point and learns a renderer that interpolates between points using an attention mechanism. This interpolation is based on the local point configurations, for example, the relative distances between points, with many variations observed from different rays during training. This variability enables attention-based methods to effectively generalize to unseen point configurations and find a promising interpolation of the points after editing, as shown on the right.

extents, using an attention mechanism [63] to interpolate between nearby points and fill gaps.

3 Preliminaries

PAPR learns a point-based representation of a 3D scene from multi-view RGB images and corresponding camera parameters. The point-based representation $\mathcal{P} = \{(\mathbf{p}_i, \mu_i)\}$, where $i \in \{1, 2, \dots, N\}$ represents a 3D scene with N neural points. Each neural point has a learnable 3D position $\mathbf{p} \in \mathbb{R}^3$ and a learnable feature vector $\mu \in \mathbb{R}^d$. Given a ray $\mathbf{r}_j$ represented by a camera center $\mathbf{o}_j \in \mathbb{R}^3$ and a ray’s direction $\mathbf{d}_j \in \mathbb{R}^3$ as the query, PAPR learns $K \ll N$ attention weights for each of the K nearest points around the ray. It then uses the attention weights to aggregate the value embedding vectors for the K points to get a feature vector $\mathbf{f}_j$ that captures the color of the ray. To map the query and key into the same feature space, PAPR uses MLPs to learn embedding vectors:

$$\mathbf{q}_j = f_{\theta_Q}(\gamma(\mathbf{d}_j)), \quad \mathbf{k}_{ij} = f_{\theta_K}([\gamma(\mathbf{h}_{i,j}), \gamma(\mathbf{t}_{i,j}), \gamma(\mathbf{p}_i)]), \quad (1)$$

where f_{θ} are the embedding MLPs, γ is the positional encoding function applied to the inputs. $\mathbf{h}_{ij}$ and $\mathbf{t}_{ij}$ are two ray-dependent point feature vectors, computed as:

$$\mathbf{p}'_{ij} = \mathbf{o}_j + \langle \mathbf{p}_i - \mathbf{o}_j, \mathbf{d}_j \rangle \cdot \mathbf{d}_j, \quad \mathbf{h}_{ij} = \mathbf{p}'_{ij} - \mathbf{o}_j, \quad \mathbf{t}_{ij} = \mathbf{p}_i - \mathbf{p}'_{ij}, \quad (2)$$

where $\mathbf{p}'_{ij}$ is the position of the point’s projection on ray $\mathbf{r}_j$. These features capture the point configuration of the neighborhood by incorporating the features from all K points around a ray. To model ray-dependent appearance, PAPR learns a ray-dependent feature vector $\mathbf{v}_{ij}$ for each point:

$$\mathbf{v}_{ij} = f_{\theta_V}([\gamma(\mathbf{h}_{i,j}), \gamma(\mathbf{t}_{i,j}), \gamma(\mu_i)]), \quad (3)$$

The ray’s feature $\mathbf{f}_j$ can then be computed by $\mathbf{f}_j = \sum_{i=1}^K w_{ij} \mathbf{v}_{ij}$, where $w_{ij} = \text{softmax}(\langle \mathbf{q}_j, \mathbf{k}_{ij} \rangle / \sqrt{d_{\mathbf{k}}})$ are the attention weights, $d_{\mathbf{k}}$ is the dimension of $\mathbf{k}_{ij}$. By spatially aggregating the feature vectors of all the rays from the same camera view, PAPR produces a feature map of the view, which is then passed through a UNet-based renderer to predict RGB image $\hat{\mathbf{I}}$. The model is end-to-end learnable by minimizing a rendering loss between $\hat{\mathbf{I}}$ and ground truth $\mathbf{I}_{\text{gt}}$:

$$\mathcal{L}_{\text{render}} = \mathcal{L}_{\text{MSE}}(\hat{\mathbf{I}}, \mathbf{I}_{\text{gt}}) + \lambda \cdot \mathcal{L}_{\text{LPIPS}}(\hat{\mathbf{I}}, \mathbf{I}_{\text{gt}}) \quad (4)$$

which is a combination of mean squared error (MSE) and LPIPS metric [76].

4 Method

4.1 3D Gaussian Splatting vs. PAPR

Compared to 3DGS, a key difference is that in PAPR, points have no spatial extent. Rather than using splats to fill gaps between points, PAPR’s attention mechanism outputs a set of weights over the K nearest points for every ray. As attention weights are non-negative and sum up to 1, it essentially learns a convex interpolation of those points. Such interpolations are based on the local point configurations around the rays, which vary significantly across different rays observed during training. This variability enables PAPR to generalize effectively to unseen point sets after deformation.

With 3DGS, it is crucial to adjust the covariance of each affected Gaussian when editing, otherwise it may cause surface discontinuities. As shown in Fig. 2, fixing covariances can result in gaps between splats after editing. Moreover, even with adjusted covariances, it’s challenging to get them right, as adjustments vary between splats. In contrast, PAPR lacks spatial extent around points, so no spatial adjustment is needed when editing—only the point locations must be changed.

Furthermore, even if the covariances were adjusted optimally, as shown in the left of Fig. 2, the shape surface after editing can still be not as smooth as it was originally. Neither shrinking nor rotating the splats would work since the former causes gaps, while the latter makes splats stick out. This is caused by the fact that non-rigid deformations of Gaussians are not necessarily Gaussian anymore. As a result, more Gaussian splats would need to be added at locations where the non-rigidity is the greatest. In contrast, since non-rigid deformations of a point set simply produce another point set, in PAPR, it suffices to move the points without adding new ones. The interpolator learned by PAPR should adapt to the changes in point positions, providing smooth and reliable interpolation for the updated point set, thereby preserving surface continuity.

4.2 Canonical Initialization

Our framework begins by capturing the canonical geometry and appearance of the target scene. We pre-train the PAPR model on a set of static multi-view

images to extract a learned dense point cloud representation $\mathcal{P}$. This canonical initialization establishes a high-fidelity reference state, provides accurate implicit surface intersections and their associated photometric properties before any non-rigid edits are applied.

4.3 Surface Point Extraction

A key component of our framework is the extraction of differentiable surface intersection estimates from PAPR’s attention mechanism. Given a camera ray $\mathbf{r}_j = (\mathbf{o}_j, \mathbf{d}_j)$ and its associated top- K selected points $\{\mathbf{p}_k\}_{k=1}^K$ with attention weights $\{w_{kj}\}$, we define the surface intersection point for ray $\mathbf{r}_j$ as the attention-weighted average of the selected point positions:

$$\hat{\mathbf{s}}_j = \sum_{k=1}^K w_{kj} \cdot \mathbf{p}_k \quad (5)$$

As the model is only trained on the observations at the canonical state of the scene, the attention model often fails to generalize to unseen edits, resulting in holes in the surface prediction and artifacts in the renderings under deformation.

4.4 Self-Supervised Fine-Tuning Objective

To ensure consistent surface prediction under deformation, we need to enhance the robustness of PAPR’s attention model to unseen edits. A naïve solution is to fine-tune the learned PAPR model on the captures of the deformed scene, for example, in the case of dynamic scene reconstruction. However, such ground truth data are not often available. To bypass the need for ground-truth dynamic sequences, we introduce a self-supervised fine-tuning objective that utilizes analytical 3D deformation fields and creates “pseudo” supervision in the fine-tuning stage.

At each fine-tuning step, we first perform a standard forward pass on the canonical point cloud $\mathcal{P}$ with a batch of training rays, extracting both rendered RGB values and *canonical* surface points $\hat{\mathbf{s}}$ from the attention weights for each ray. We then apply an analytically defined 3D deformation field $\mathcal{D} : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ to the point cloud, yielding the deformed point cloud $\mathcal{P}_{\text{def}} = \mathcal{D}(\mathcal{P})$.

We then re-query the model under the deformed point cloud with deformed rays (Sec. 4.5) to extract predicted *deformed* surface points $\hat{\mathbf{s}}_{\text{pred}}$, and compute a geometric loss between $\hat{\mathbf{s}}_{\text{pred}}$, and the deformed *canonical* surface points $\mathcal{D}(\hat{\mathbf{s}}_j)$, using the identical deformation field $\mathcal{D}$:

$$\mathcal{L}_{\text{geo}} = \frac{1}{|\mathcal{F}|} \sum_{j \in \mathcal{F}} \|\hat{\mathbf{s}}_{\text{pred},j} - \mathcal{D}(\text{sg}[\hat{\mathbf{s}}_j])\|^2 \quad (6)$$

where $\mathcal{F}$ denotes the set of ray indices and $\text{sg}[\cdot]$ is the stop-gradient operator, which detaches the *canonical* surface points $\hat{\mathbf{s}}$ from the computation graph before applying the deformation $\mathcal{D}$. This prevents gradients from flowing back through the canonical extraction and destabilizing the already-converged representation,

ensuring only the deformed-state predictions $\hat{\mathbf{s}}_{\text{pred}}$ are optimized. Additionally, we enforce photometric consistency on the deformed rendering through a texture loss $\mathcal{L}_{\text{tex}} = \|\hat{I}_{\text{def}} - \hat{I}_{\text{can}}\|^2$, which anchors the appearance of the deformed state to the canonical rendering without requiring ground-truth deformed images.

4.5 Deformation Modes

We introduce two complementary deformation modes that determine how camera rays interact with the deformed geometry, each addressing a distinct failure mode:

Points-only mode. The point cloud and surface points are deformed, but the camera ray origins remain at their original positions. Foreground ray directions are recomputed to point toward the deformed ray termination points. This mode preserves the original camera viewpoint structure, ensuring robust rendering quality from distant views where the deformation is observed globally.

Rays-and-points mode. In addition to deforming the point cloud and surface points, the ray origins are repositioned near the surface and then deformed by the same field. Ray directions are recomputed from the deformed origins toward the deformed termination points. This mode addresses the occlusion problem: when the point cloud deforms, the original ray directions may no longer correctly query the deformed surface due to self-occlusion from displaced geometry. By diversifying the set of ray directions the model encounters during training, this mode significantly enhances robustness to complex deformations.

In practice, we use both modes simultaneously.

4.6 Joint Loss Formulation

The total training objective combines geometric alignment on the deformed state with appearance anchoring on the canonical state:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rgb}} + \lambda_{\text{geo}} \cdot \mathcal{L}_{\text{geo}} + \lambda_{\text{tex}} \cdot \mathcal{L}_{\text{tex}} \quad (7)$$

where $\mathcal{L}_{\text{rgb}}$ is the canonical photometric rendering loss (Eq. 4), $\mathcal{L}_{\text{geo}}$ is the self-supervised geometric loss (Eq. 6), and $\mathcal{L}_{\text{tex}}$ is the texture consistency loss, all defined in Sec. 4.4.

5 Experiments

5.1 Datasets and Baselines

To evaluate the structural resilience and rendering fidelity of our self-supervised framework, we test against both proxy-based and proxy-free point editing methods on the Neural Editor dataset [14] and dynamic scene subsets from Objaverse [16] introduced by PAPR-in-Motion [52], a recent method that extends PAPR to synthesize smooth point-level 3D scene interpolations between different scene

states without intermediate supervision. For these synthetic datasets, ground truth deformed states are available, enabling both qualitative and quantitative benchmark comparisons. We follow the scene selection and evaluation protocols from [14] for Neural Editor and PAPR-in-Motion [52] for Objaverse, evaluating on scenes containing complex non-rigid transformations and part-level motions. Additionally, we qualitatively evaluate our method on real-world scenes from the DTU and Mip-NeRF 360 datasets as we lack ground truth deformation fields and renderings of the deformed scenes for these real-world datasets. We utilize a variety of analytical 3D deformation fields during our self-supervised training across all datasets, such as global twisting, scaling, dilating, and bending along random directions and varying degrees.

For proxy-based baselines, we compare with Deforming-NeRF [70], Neural Editor [14] (cage-based), and Mani-GS [24] (mesh-based). For proxy-free baselines, we compare with SC-GS [30] and the vanilla PAPR [77] model to explicitly demonstrate the robustness provided by our framework. SC-GS and Mani-GS represent two common ways to deform the covariance of the Gaussians (which must be deformed, otherwise Gaussians will easily extrude out of the surface); SC-GS uses the ARAP geometry regularizer, and Mani-GS uses a mesh as proxy geometry. In contrast, our method inherently treats each point as infinitesimal, so we can directly update the point positions without worrying about the shape of the points. As training Neural Editor requires around 7 days per scene on an NVIDIA RTX 3090, we focused our evaluation on quantitative comparisons on the Neural Editor dataset, given limited resources. For a fair comparison we apply identical deformation fields across all methods and evaluate the resulting visual and structural fidelity. In our method, the fine-tuning requires only ~ 500 steps ($< 1\text{min}$)—roughly $1/500$ of the pre-training duration—using the same learning rates and batch size, making it a lightweight post-processing step.

5.2 Qualitative Results

Fig. 3 and Fig. 5 show the qualitative comparison between P-CORE and the baselines on the Neural Editor and Objaverse datasets. To align the deformations across different methods for fairness, we first manually deform the cage from Deforming-NeRF, and then use the cage to deform the mesh or the control points of other methods through cage-based deformation.

As shown in the figures, proxy-based approaches like Mani-GS often struggle with part-level manipulations and complex topological changes. Because they are bound by a rigid mesh, they fail to separate distinct parts or introduce blocky artifacts when the underlying topology is severely stretched. In contrast, proxy-free methods like SC-GS do not rely on geometry proxies, making them more adaptable to various types of deformations. However, SC-GS severely fails to preserve surface continuity after editing, leading to pervasive holes and "extruded Gaussians" around the boundaries of the edited shapes. Furthermore, while the base PAPR model handles minor interpolations well, it still exhibits noticeable tearing and structural collapse under large non-rigid edits.



Fig. 3: Qualitative comparison on the Neural Editor dataset.

In contrast, our P-CORE framework significantly enhances structural robustness. It drastically reduces holes, tearing, and other artifacts, consistently preserving perfectly continuous surfaces across all scenes even under severe deformation.

Additionally, we demonstrate the versatility of our method on complex real-world captures. As shown in Fig. 4, P-CORE extends robustly to the DTU and Mip-NeRF 360 datasets, supporting multiple sequential edits without degrading texture quality or introducing surface tearing.

5.3 Quantitative Results

To quantitatively evaluate our zero-shot editing performance, we compare our method against all baselines on scenes from the Neural Editor [14] and Objaverse [16] datasets where ground truth deformed states are available. By applying identical deformation fields to the points or proxies, we ensure a fair comparison across all methods.



Fig. 4: Qualitative editing results on real-world scenes from the DTU and Mip-NeRF 360 datasets.

Table 1: Average quantitative performance on the Neural Editor and Objaverse datasets across all scenes. **Bold** indicates the best and underline indicates the second best.

Method	Neural Editor [14]				Objaverse [16]			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# Primitives	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# Primitives
Deforming-NeRF [70]	14.71	0.722	0.197	-	15.89	0.773	0.180	-
Neural Editor [14]	25.62	<u>0.939</u>	0.078	$\sim 1\text{M}$	-	-	-	-
SC-GS [30]	18.55	0.832	0.123	$\sim 236\text{K}$	25.29	0.929	<u>0.057</u>	$\sim 228\text{K}$
Mani-GS [24]	20.59	0.875	0.116	$\sim 1.2\text{M}$	<u>26.04</u>	<u>0.946</u>	0.069	$\sim 362\text{K}$
PAPR [77]	24.22	0.930	<u>0.066</u>	30K	23.15	0.940	0.058	30K
Ours	<u>25.31</u>	0.942	0.054	30K	28.30	0.965	0.039	30K

Table 1 summarizes the average quantitative performance. P-CORE achieves state-of-the-art rendering quality across both datasets across all metrics (PSNR, SSIM, and LPIPS) on the Objaverse dataset, and either matches or outperforms the baselines on the Neural Editor dataset, with a much smaller number of primitives (30K v.s. 1M).

5.4 Ablation Study

To evaluate the contribution of each component in our self-supervised fine-tuning framework, we conduct an ablation study on the Lego scene under a $2\times$ uniform scaling deformation. Fig. 6 provides a qualitative comparison of the rendered novel views and depth maps across different configurations.

Vanilla PAPR (without fine-tuning) exhibits obvious holes in both the novel views and depth maps, confirming the base representation’s inability to maintain surface continuity under large deformations. **Without the rays-and-points mode**, many holes still remain in both the depth and rendering, as the ray directions are not diverse enough for the attention model to properly generalize to the deformed geometry. **Without the points-only mode**, the depth around the boundaries of the object becomes inaccurate, and there are visible artifacts around the boundaries in the rendering as well, since the fine-tuning overfits to the close viewpoints sampled in the rays-and-points mode. **Without the texture consistency loss** ($\mathcal{L}_{\text{tex}}$), the depth map has far fewer holes, but the rendering is



Fig. 5: Qualitative comparison on scenes from Objaverse.

noticeably blurred. **Our full method** with all components produces a hole-free depth map and a high-fidelity rendering with preserved details, demonstrating that all proposed components are essential for robust deformation.

5.5 Application: Skinning Weights Optimization

A downstream application of our self-supervised framework is to optimize the skinning weights of the parameterized scenes without a ground truth motion sequence.

Specifically, we extract $M(=128)$ sparse control anchors $\{\mathbf{a}_m\}_{m=1}^M$ from the canonical dense point cloud using Farthest Point Sampling (FPS). We parameterize the dense point cloud using a Neural Blend Skinning (NBS) approach, where

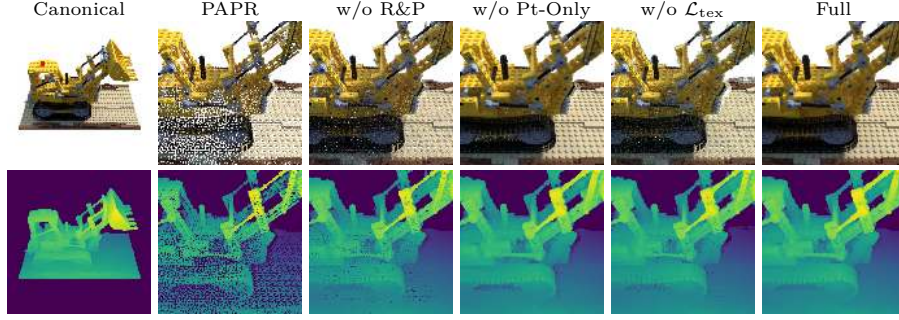


Fig. 6: Ablation study on the Lego scene under $2\times$ scaling. Top: rendering. Bottom: depth. From left to right: canonical (undeformed), vanilla PAPR, without rays-and-points mode, without points-only mode, without texture loss $\mathcal{L}_{\text{tex}}$, and our full pipeline.

an anchor skinning MLP f_ϕ with positional encoding takes a normalized 3D point coordinate and outputs logits over all M anchors: $\ell(\mathbf{p}) = f_\phi(\gamma(\mathbf{p})) \in \mathbb{R}^M$. For each point $\mathbf{p}$ to be deformed, we find its $K_{\text{nn}} (= 8)$ nearest anchors by Euclidean distance, gather the corresponding logits, and apply softmax to obtain sparse skinning weights: $w_k = \text{softmax}(\ell_{m_k}(\mathbf{p}))$.

To derive per-anchor rigid transformations from an analytical deformation $\mathcal{D}$, we estimate the Jacobian $\mathbf{J}_m = \nabla \mathcal{D}(\mathbf{a}_m)$ via finite differences, extract the closest rotation matrix $\mathbf{R}_m$ via SVD projection, and recover the translation $\mathbf{t}_m = \mathcal{D}(\mathbf{a}_m) - \mathbf{R}_m \mathbf{a}_m$. The deformed position is then predicted via Linear Blend Skinning:

$$\mathbf{p}' = \sum_{k=1}^{K_{\text{nn}}} w_k \cdot (\mathbf{R}_{m_k} \mathbf{p} + \mathbf{t}_{m_k}) \quad (8)$$

This NBS parameterization replaces the direct analytical deformation for the point cloud and surface points within our self-supervised framework, while camera rays are still deformed analytically. As the pipeline is end-to-end differentiable, during the self-supervised fine-tuning, we jointly optimize these skinning MLP weights alongside the neural renderer. At inference time, users perform edits by simply repositioning the sparse anchor points.

Table 2 compares our optimized NBS weights against distance-based Gaussian-kernel RBF weights [30], where $\hat{w}_{jk} = \exp(-d_{jk}^2/2\sigma^2)$ with $\sigma=1$, normalized to sum to one. For the geometry evaluation, we compare the deformed full point cloud derived from the deformed anchor points with the ground truth deformed mesh. For a fair comparison we use the same set of anchor points for both methods. As shown, our NBS consistently outperforms the RBF baseline across all rendering and geometry metrics, confirming that self-supervised deformation augmentation yields spatially discriminative skinning weights that generalize and better preserve geometry detail during editing. Fig. 7 shows qualitative editing results on two Objaverse scenes.

Table 2: Ablation on skinning weight estimation, averaged over 6 Objaverse scenes. Both methods use the same FPS anchors; the dense point cloud is deformed via anchor-driven RBF interpolation and compared against ground-truth end-state views and meshes (normalised to $[-1, 1]^3$).

Skinning Weights	Rendering Quality			Geometry Quality			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	CD $\downarrow$	EMD $\downarrow$	F _{0.01} $\uparrow$	F _{0.02} $\uparrow$
RBF [30]	25.32	0.945	0.054	0.077	0.307	0.409	0.626
NBS (Ours)	26.46	0.952	0.050	0.077	0.303	0.438	0.640

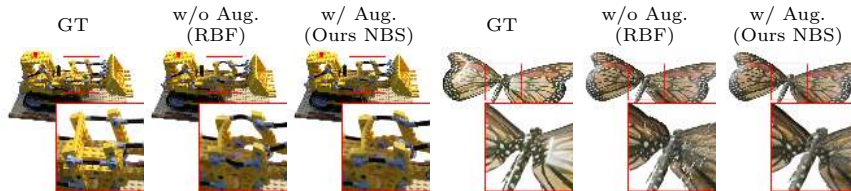


Fig. 7: Qualitative comparison of proxy-free sparse control editing on the Lego and Butterfly scenes. Without self-supervised augmentation, the RBF-deformed renderings exhibit unfaithful deformation and surface discontinuities. With our self-supervised fine-tuning, the edited geometry is better aligned with the ground-truth target.

6 Conclusion and Future Work

In this paper, we presented a self-supervised surface consistency framework that addresses the fundamental tension in point-based neural rendering: the lack of fixed connectivity that makes these representations flexible for reconstruction also causes holes and surface discontinuities under large deformations. By generating random deformations and enforcing that surface prediction commutes with deformation, our approach eliminates the need for ground truth dynamic data or geometry proxies. We incorporated this framework into PAPR, an attention-based neural point renderer whose learned interpolation kernel can adapt to large deformations without requiring addition or removal of points, and demonstrated significant improvements on both synthetic benchmarks and real-world scenes, as well as a downstream application to Neural Blend Skinning (NBS). Future work includes integrating physically-based simulations, more sophisticated multi-level editing controls, and extending the framework to other point-based architectures.

References

1. Alexa, M., Behr, J., Cohen-Or, D., Fleishman, S., Levin, D., Silva, C.T.: Computing and rendering point set surfaces. *IEEE Trans. Vis. Comput. Graph.* **9**, 3–15 (2003), <https://api.semanticscholar.org/CorpusID:7929773>
2. Alexa, M., Cohen-Or, D., Levin, D.: As-rigid-as-possible shape interpolation. *Proceedings of the 27th annual conference on Computer graphics and interactive techniques* (2000), <https://api.semanticscholar.org/CorpusID:3337513>
3. Aliev, K.A., Ulyanov, D., Lempitsky, V.S.: Neural point-based graphics. In: *European Conference on Computer Vision* (2019)
4. Angelidis, A., Cani, M.P., Wyvill, G., King, S.: Swirling-sweepers: constant volume modeling. In: *ACM SIGGRAPH 2004 Sketches*, p. 40 (2004)
5. Aubert, F., Bechmann, D.: Volume-preserving space deformation. *Computers & Graphics* **21**(5), 625–639 (1997)
6. Bechmann, D., Dubreuil, N.: Animation through space and time based on a space deformation model. *The Journal of Visualization and Computer Animation* **4**(3), 165–184 (1993)
7. Bechmann, D., Dubreuil, N.: Order-controlled free-form animation. *The Journal of Visualization and computer animation* **6**(1), 11–32 (1995)
8. Bechmann, D., Gerber, D.: Arbitrary shaped deformations with dogme. *The Visual Computer* **19**(2), 175–186 (2003)
9. Bian, W., Huang, Z., Shi, X., Li, Y., Wang, F.Y., Li, H.: Gs-dit: Advancing video generation with pseudo 4d gaussian fields through efficient dense 3d point tracking. *ArXiv abs/2501.02690* (2025), <https://api.semanticscholar.org/CorpusID:275336281>
10. Borrel, P., Bechmann, D.: Deformation of n-dimensional objects. In: *Proceedings of the first ACM symposium on Solid modeling foundations and CAD/CAM applications*. pp. 351–369 (1991)
11. Borrel, P., Rappoport, A.: Simple constrained deformations for geometric modeling and interactive design. *ACM Transactions on Graphics (TOG)* **13**(2), 137–155 (1994)
12. Botsch, M., Kobbelt, L.: Real-time shape editing using radial basis functions. *Computer Graphics Forum* **24** (2005), <https://api.semanticscholar.org/CorpusID:11339322>
13. Chang, J.H.R., Chen, W.Y., Ranjan, A., Yi, K.M., Tuzel, O.: Pointersect: Neural rendering with cloud-ray intersection. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* pp. 8359–8369 (2023)
14. Chen, J.K., Lyu, J., Wang, Y.X.: Neuraleditor: Editing neural radiance fields via manipulating point clouds. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* pp. 12439–12448 (2023), <https://api.semanticscholar.org/CorpusID:258479756>
15. Decaudin, P.: Geometric deformation by merging a 3d-object with a simple shape. In: *Graphics Interface*. vol. 96, pp. 55–60 (1996)
16. Deitke, M., Schwenk, D., Salvador, J., Weihs, L., Michel, O., VanderBilt, E., Schmidt, L., Ehsani, K., Kembhavi, A., Farhadi, A.: Objaverse: A universe of annotated 3d objects. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* pp. 13142–13153 (2022), <https://api.semanticscholar.org/CorpusID:254685588>
17. Erler, P., Guerrero, P., Ohrhallinger, S., Mitra, N.J., Wimmer, M.: Points2surf learning implicit surfaces from point clouds. In: *European Conference on Computer Vision* (2020), <https://api.semanticscholar.org/CorpusID:226298441>

18. Feng, H., Zhang, J., Wang, Q., Ye, Y., Yu, P., Black, M.J., Darrell, T., Kanazawa, A.: St4rtrack: Simultaneous 4d reconstruction and tracking in the world. ArXiv **abs/2504.13152** (2025), <https://api.semanticscholar.org/CorpusID:277857146>
19. Feng, J., Ma, L., Peng, Q.: A new free-form deformation through the control of parametric surfaces. *Computers & Graphics* **20**(4), 531–539 (1996)
20. Feng, J., Shao, J., Jin, X., Peng, Q., Forrest, A.R.: Multiresolution free-form deformation with subdivision surface of arbitrary topology. *The Visual Computer* **22**, 28–42 (2006)
21. Gain, J.E., Bechmann, D.: A survey of spatial deformation from a user-centered perspective. *ACM Trans. Graph.* **27**, 107:1–107:21 (2008), <https://api.semanticscholar.org/CorpusID:12812109>
22. Gao, L., Lai, Y.K., Yang, J., Zhang, L.X., Xia, S., Kobbelt, L.: Sparse data driven mesh deformation. *IEEE transactions on visualization and computer graphics* **27**(3), 2085–2100 (2019)
23. Gao, L., Yang, J., Zhang, B.T., Sun, J., Yuan, Y.J., Fu, H., Lai, Y.K.: Mesh-based gaussian splatting for real-time large-scale deformation. ArXiv **abs/2402.04796** (2024), <https://api.semanticscholar.org/CorpusID:267523424>
24. Gao, X., Li, X., Zhuang, Y., Zhang, Q., Hu, W., Zhang, C., Yao, Y., Shan, Y., Quan, L.: Mani-gs: Gaussian splatting manipulation with triangular mesh. ArXiv **abs/2405.17811** (2024), <https://api.semanticscholar.org/CorpusID:270068019>
25. Gu’edon, A., Lepetit, V.: Gaussian frosting: Editable complex radiance fields with real-time rendering. ArXiv **abs/2403.14554** (2024), <https://api.semanticscholar.org/CorpusID:268553521>
26. Han, X., Tian, R., Tong, Y., Yu, F., Liu, D., Zhang, Y.: Arap-gs: Drag-driven as-rigid-as-possible 3d gaussian splatting editing with diffusion prior (2025), <https://arxiv.org/abs/2504.12788>
27. Hsu, W.M., Hughes, J.F., Kaufman, H.: Direct manipulation of free-form deformations. *ACM Siggraph Computer Graphics* **26**(2), 177–184 (1992)
28. Hu, S.M., Zhang, H., Tai, C.L., Sun, J.G.: Direct manipulation of ffd: efficient explicit solutions and decomposable multiple point constraints. *Visual Computer* **17**(6), 370–379 (2001)
29. Huang, J., Yu, H.: Gsdeformer: Direct cage-based deformation for 3d gaussian splatting. ArXiv **abs/2405.15491** (2024), <https://api.semanticscholar.org/CorpusID:270045641>
30. Huang, Y.H., tian Sun, Y., Yang, Z., Lyu, X., Cao, Y.P., Qi, X.: Sc-gs: Sparse-controlled gaussian splatting for editable dynamic scenes. ArXiv **abs/2312.14937** (2023), <https://api.semanticscholar.org/CorpusID:266551723>
31. Jambon, C., Kerbl, B., Kopanas, G., Diolatzis, S., Leimkühler, T., Drettakis, G.: Nerfshop. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* **6**, 1 – 21 (2023), <https://api.semanticscholar.org/CorpusID:258718613>
32. Jiang, Y., Yu, C., Xie, T., Li, X., Feng, Y., Wang, H., Li, M., Lau, H., Gao, F., Yang, Y., Jiang, C.: Vr-gs: A physical dynamics-aware interactive gaussian splatting system in virtual reality. ArXiv **abs/2401.16663** (2024), <https://api.semanticscholar.org/CorpusID:267320640>
33. Jin, X., Li, V.: Three-dimensional deformation using directional polar coordinates. *Journal of Graphics Tools* **5**(2), 15–24 (2000)
34. Ju, T., Schaefer, S., Warren, J.D.: Mean value coordinates for closed triangular meshes. *ACM SIGGRAPH 2005 Papers* (2005), <https://api.semanticscholar.org/CorpusID:3404588>

35. Kazhdan, M.M., Bolitho, M., Hoppe, H.: Poisson surface reconstruction. In: Eurographics Symposium on Geometry Processing (2006), <https://api.semanticscholar.org/CorpusID:14224>
36. Kazhdan, M.M., Hoppe, H.: Screened poisson surface reconstruction. *ACM Trans. Graph.* **32**, 29:1–29:13 (2013), <https://api.semanticscholar.org/CorpusID:1371704>
37. Kerbl, B., Kopanas, G., Leimkuehler, T., Drettakis, G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)* **42**, 1 – 14 (2023)
38. Kobayashi, K.G., Ootsubo, K.: t-ffd: free-form deformation by using triangular mesh. In: Proceedings of the eighth ACM symposium on Solid modeling and applications. pp. 226–234 (2003)
39. Kopanas, G., Philip, J., Leimkuehler, T., Drettakis, G.: Point-based neural rendering with per-view optimization. *Computer Graphics Forum* **40** (2021)
40. Lassner, C., Zollhöfer, M.: Pulsar: Efficient sphere-based neural rendering. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 1440–1449 (2021)
41. Lee, S.Y., Chwa, K.Y., Shin, S.Y.: Image metamorphosis using snakes and free-form deformations. In: Proceedings of the 22nd annual conference on Computer graphics and interactive techniques. pp. 439–448 (1995)
42. Lei, H.: Offsetopt: Explicit surface reconstruction without normals. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 11729–11738 (2025), <https://api.semanticscholar.org/CorpusID:277151096>
43. Levin, D.: Mesh-independent surface interpolation (2004), <https://api.semanticscholar.org/CorpusID:119494631>
44. Lipman, Y., Sorkine, O., Alexa, M., Cohen-Or, D., Levin, D., Rössl, C., Seidel, H.P.: Laplacian framework for interactive mesh editing. *International Journal of Shape Modeling* **11**(01), 43–61 (2005)
45. Liu, R., Xiang, J., Zhao, B., Zhang, R., Yu, J., Zheng, C.: Neural impostor: Editing neural radiance fields with explicit shape manipulation. *Computer Graphics Forum* **42** (2023), <https://api.semanticscholar.org/CorpusID:263828751>
46. Ma, B., Han, Z., Liu, Y.S., Zwicker, M.: Neural-pull: Learning signed distance functions from point clouds by learning to pull space onto surfaces. *ArXiv abs/2011.13495* (2020), <https://api.semanticscholar.org/CorpusID:227209266>
47. Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: Nerf: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* **65**, 99–106 (2020)
48. Moccozet, L., Magnenat-Thalmann, N.: Dirichlet free-form deformations and their application to hand simulation. *Proceedings. Computer Animation '97 (Cat. No.97TB100120)* pp. 93–102 (1997), <https://api.semanticscholar.org/CorpusID:12257663>
49. Müller, M., Heidelberger, B., Teschner, M., Gross, M.H.: Meshless deformations based on shape matching. *ACM SIGGRAPH 2005 Papers* (2005), <https://api.semanticscholar.org/CorpusID:3401588>
50. Müller, T., Evans, A., Schied, C., Keller, A.: Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)* **41**(4), 1–15 (2022)
51. Ost, J., Laradji, I.H., Newell, A., Bahat, Y., Heide, F.: Neural point light fields. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 18398–18408 (2021)

52. Peng, S., Zhang, Y., Li, K.: Papr in motion: Seamless point-level 3d scene interpolation. 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 21007–21016 (2024), <https://api.semanticscholar.org/CorpusID:270371843>
53. Peng, Y., Yan, Y., Liu, S., Cheng, Y., Guan, S., Pan, B., Zhai, G., Yang, X.: Cagenerf: Cage-based neural radiance field for generalized 3d deformation and animation. In: Neural Information Processing Systems (2022), <https://api.semanticscholar.org/CorpusID:258509626>
54. Raffin, R., Neveu, M., Jaar, F.: Curvilinear displacement of free-form-based deformation. *The Visual Computer* **16**(1), 38–46 (2000)
55. Rakhimov, R., Ardelean, A.T., Lempitsky, V.S., Burnaev, E.: Npbg++: Accelerating neural point-based graphics. *ArXiv abs/2203.13318* (2022)
56. Rückert, D., Franke, L., Stamminger, M.: Adop: Approximate differentiable one-pixel point rendering. *ACM Trans. Graph.* **41**, 99:1–99:14 (2021)
57. Rupprecht, D., Müller, H.: Free form deformation with scattered data interpolation methods. In: *Geometric Modeling* (1993), <https://api.semanticscholar.org/CorpusID:1816929>
58. Singh, K., Fiume, E.: Wires: a geometric deformation technique. In: *Proceedings of the 25th annual conference on Computer graphics and interactive techniques*. pp. 405–414 (1998)
59. Sorkine, O.: Laplacian mesh processing. *Eurographics (State of the Art Reports)* **4**(4), 1 (2005)
60. Sorkine, O., Alexa, M.: As-rigid-as-possible surface modeling. In: *Symposium on Geometry processing*. vol. 4, pp. 109–116. Citeseer (2007)
61. Sorkine, O., Cohen-Or, D., Lipman, Y., Alexa, M., Rössl, C., Seidel, H.P.: Laplacian surface editing. In: *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. pp. 175–184 (2004)
62. Sorkine-Hornung, O., Alexa, M.: As-rigid-as-possible surface modeling. In: *Eurographics Symposium on Geometry Processing* (2007), <https://api.semanticscholar.org/CorpusID:11952089>
63. Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Neural Information Processing Systems* (2017)
64. Von Funck, W., Theisel, H., Seidel, H.P.: Vector field based shape deformations. *ACM Transactions on Graphics (ToG)* **25**(3), 1118–1125 (2006)
65. Waczyńska, J., Borycki, P., Tadeja, S.K., Tabor, J., Spurek, P.: Games: Mesh-based adapting and modification of gaussian splatting. *ArXiv abs/2402.01459* (2024), <https://api.semanticscholar.org/CorpusID:267406241>
66. Wang, C., He, M., Chai, M., Chen, D., Liao, J.: Mesh-guided neural implicit field editing. *ArXiv abs/2312.02157* (2023), <https://api.semanticscholar.org/CorpusID:265609951>
67. Wiles, O., Gkioxari, G., Szeliski, R., Johnson, J.: Synsin: End-to-end view synthesis from a single image. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 7465–7475 (2019)
68. Wu, G., Yi, T., Fang, J., Xie, L., Zhang, X., Wei, W., Liu, W., Tian, Q., Wang, X.: 4d gaussian splatting for real-time dynamic scene rendering. 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 20310–20320 (2023), <https://api.semanticscholar.org/CorpusID:263908793>
69. Xu, Q., Xu, Z., Philip, J., Bi, S., Shu, Z., Sunkavalli, K., Neumann, U.: Point-nerf: Point-based neural radiance fields. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 5428–5438 (2022)

70. Xu, T., Harada, T.: Deforming radiance fields with cages. ArXiv **abs/2207.12298** (2022)
71. Yang, B., Bao, C., Zeng, J., Bao, H., Zhang, Y., Cui, Z., Zhang, G.: Neumesh: Learning disentangled neural mesh-based implicit field for geometry and texture editing. ArXiv **abs/2207.11911** (2022), <https://api.semanticscholar.org/CorpusID:251040986>
72. Yifan, W., Aigerman, N., Kim, V.G., Chaudhuri, S., Sorkine-Hornung, O.: Neural cages for detail-preserving 3d deformations. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 75–83 (2020)
73. Yuan, Y.J., Lai, Y.K., Wu, T., Gao, L., Liu, L.: A revisit of shape editing techniques: From the geometric to the neural viewpoint. *Journal of Computer Science and Technology* **36**, 520 – 554 (2021), <https://api.semanticscholar.org/CorpusID:232092888>
74. Yuan, Y.J., Sun, Y.T., Lai, Y.K., Ma, Y., Jia, R., Gao, L.: Nerf-editing: Geometry editing of neural radiance fields. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 18332–18343 (2022)
75. Zhang, Q., Baek, S.H., Rusinkiewicz, S., Heide, F.: Differentiable point-based radiance fields for efficient view synthesis. *SIGGRAPH Asia 2022 Conference Papers* (2022)
76. Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition pp. 586–595 (2018)
77. Zhang, Y., Peng, S., Moazenipourasil, S.A., Li, K.: PAPR: Proximity attention point rendering. In: Thirty-seventh Conference on Neural Information Processing Systems (2023)
78. Zhang, Y., Zheng, J., Cai, Y.: Proxy-driven free-form deformation by topology-adjustable control lattice. *Computers & Graphics* **89**, 167–177 (2020)
79. Zheng, C., Chang Lin, W., Xu, F.: Editablenerf: Editing topologically varying neural radiance fields by key points. 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) pp. 8317–8327 (2022), <https://api.semanticscholar.org/CorpusID:254408978>
80. Zhou, K., Hong, L., Xie, E., Yang, Y., Li, Z., Zhang, W.: Serf: Fine-grained interactive 3d segmentation and editing with radiance fields. ArXiv **abs/2312.15856** (2023), <https://api.semanticscholar.org/CorpusID:266551558>
81. Zuo, Y., Deng, J.: View synthesis with sculpted neural points. ArXiv **abs/2205.05869** (2022)
82. Zwicker, M., Pauly, M., Knoll, O., Gross, M.H.: Pointshop 3d: an interactive system for point-based surface editing. *Proceedings of the 29th annual conference on Computer graphics and interactive techniques* (2002), <https://api.semanticscholar.org/CorpusID:1828662>

A Extended Comparison with Neural Editor

We provide qualitative comparisons with Neural Editor on both datasets, along with the full quantitative table and a runtime analysis for convenience.

A.1 Quantitative Results

Table 3 reproduces Table 1 from the main paper for reference. Our method outperforms Neural Editor on Objaverse across all three metrics while using only 30K points. Beyond training cost, Neural Editor also incurs heavy overhead at inference: rendering a single deformed point cloud requires ~ 23 minutes of preprocessing on an NVIDIA RTX 3090—including KD-tree construction, mesh normal recomputation, and infinitesimal surface transformation—whereas our method requires no preprocessing of any kind.

Table 3: Average quantitative performance on the Neural Editor and Objaverse datasets across all scenes, including Neural Editor baseline results on Objaverse. **Bold** indicates the best and underline indicates the second best.

Method	Neural Editor [14]				Objaverse [16]			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# Primitives	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# Primitives
Deforming-NeRF [70]	14.71	0.722	0.197	-	15.89	0.773	0.180	-
Neural Editor [14]	25.62	<u>0.939</u>	0.078	$\sim 1\text{M}$	<u>28.02</u>	<u>0.956</u>	<u>0.048</u>	$\sim 1\text{M}$
SC-GS [30]	18.55	0.832	0.123	$\sim 236\text{K}$	25.29	0.929	0.057	$\sim 228\text{K}$
Mani-GS [24]	20.59	0.875	0.116	$\sim 1.2\text{M}$	26.04	0.946	0.069	$\sim 362\text{K}$
PAPR [77]	24.22	0.930	<u>0.066</u>	30K	23.15	0.940	0.058	30K
Ours	<u>25.31</u>	0.942	0.054	30K	28.30	0.965	0.039	30K

A.2 Qualitative Comparison with Neural Editor

Figures 8 and 9 present side-by-side qualitative comparisons between our method and Neural Editor on the Neural Editor and Objaverse datasets, respectively. As shown, both methods produce visually faithful deformations. However, Neural Editor occasionally exhibits holes or color bleeding in fine-detail regions, whereas our method preserves surface continuity.

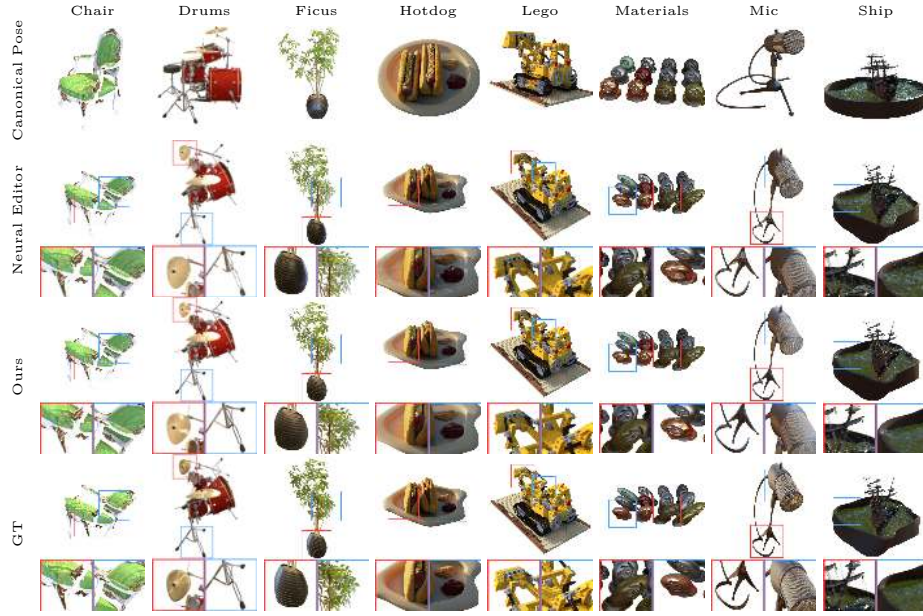


Fig. 8: Qualitative comparison between our method and Neural Editor on the Neural Editor dataset [14].

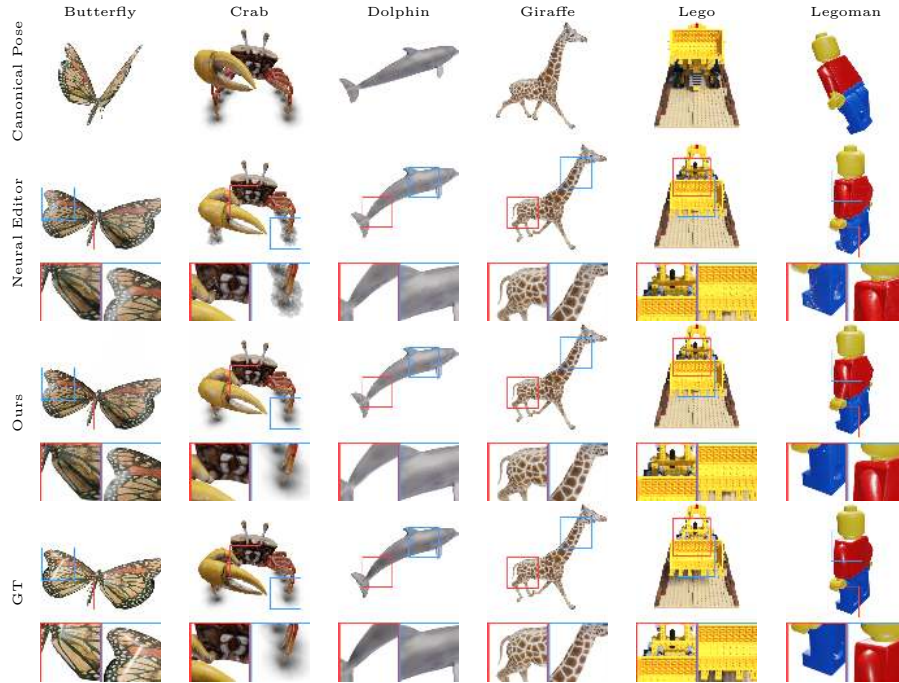


Fig. 9: Qualitative comparison between our method and Neural Editor on scenes from Objaverse [16, 52].